
UNIT 7 STRUCTURED QUERY LANGUAGE

- 7.0 Introduction
- 7.1 Objectives
- 7.2 Introduction to SQL
- 7.3 Data Definition Language
- 7.4 Data Manipulation Language
 - 7.4.1 Data insertion, Updating and Deletion
 - 7.4.2 Data Retrieval
- 7.5 GROUP BY Clause and Aggregate functions
- 7.6 Data Control Language
- 7.7 Summary
- 7.8 Solutions/Answers

7.0 INTRODUCTION

As discussed in the previous Units, a relational database system consists of relations, which are normalised to minimise redundancy. The normalisation process involves the concepts of functional dependency (FD), multi-valued dependency (MVD) and join dependency (JD). The normalisation results in the decomposition of tables into smaller but non-redundant relations. After designing the normalised database system, you would like to implement it by using software called relational database management system (RDBMS). RDBMSs were designed to manage relational data. Some of these RDBMSs are – Oracle, DB2, SQL Server, MySQL, etc. All these RDBMSs are designed and developed by different organisations and use different data storage formats. A structured query language (SQL) is one of the standards, which was created for the transfer of information from a RDBMS. The SQL consists of three basic languages, viz. Data Definition Language (DDL), which is used for defining the structure of the relations in the form of SQL tables and indexes; Data Manipulation Language, which is used for input, modification and deletion of information in SQL tables; and Data Control Language, which is used for controlling the access rights on the data of SQL tables and indexes. This unit introduces these three languages and the basic set of commands used in SQL.

You must learn SQL to use database technologies, therefore, you are advised to go through this unit very carefully. You must practice the concepts learnt in this unit on a commercial RDBMS.

7.1 OBJECTIVES

After going through this unit, you should be able to:

- create SQL objects (tables, indexes, etc.) from a database schema;
- insert data into database tables using SQL commands;
- retrieve data from a database using SQL queries;
- use the Group By and Having clauses of SQL;
- Using aggregate functions of SQL;
- Create access rights on different database objects using SQL.

7.2 INTRODUCTION TO SQL

The Structured Query Language (SQL) was initially designed at IBM by D D Chamberlin and R F Boyce for the relational model that was proposed in 1970 by E F Codd. SQL was designed as a fourth-generation programming language to produce multiple record output as a result of a single command. In addition, SQL commands did not require the specification of indexes or other information for data retrieval. Later, SQL was standardised by the American National Standard Institute (ANSI) and a number of standards of SQL have been created, starting from SQL:86, which was proposed in 1986, SQL:92, SQL:99 and SQL:2003. Several modifications have been proposed in SQL:2003 in the years 2006-2019. One of the key advantages of using SQL is that it is *the query language* of most RDBMSs. Thus, it facilitates the migration of a database from one RDBMS to another RDBMS. However, some differences can be found in the SQL implementation on several RDBMSs. This unit covers some of the basic features of SQL only.

Why did SQL become popular? One of the major reasons for SQL's popularity is that it allows you to write queries without specifying how the query would be executed. For example, in case you want to JOIN three relations or tables, say A, B, C, then SQL allows you to join these tables without specifying the sequence of joining. The decision about how to execute the joins, e.g. (A JOIN B) JOIN C or A JOIN (B JOIN C), what indexes and views are to be used, etc.; are left to the query parser, translator, and query optimiser of RDBMS. In addition, the syntax of SQL is closer to the English language. Thus, SQL queries are simpler to write and run. The following are the features of SQL in the context of RDBMS:

- SQL is non-procedural, as in SQL you just need to say what information is required by you and NOT how that information is to be acquired from the database.
- The syntax of SQL is closer to the English Language; thus, it is very easy to comprehend.
- In general, the output of a SQL query may be a single record or a group of records.
- An interesting feature of SQL is that it can be used at different levels of ANSI's three-level architecture of database system.

SQL includes the following three sub-languages:

- Data Definition Language (DDL): The basic focus of DDL is the commands that can be used to convert database schema into SQL tables, indexes, etc., or change of structure of the tables. These commands also allow you to define the constraints on the attributes of a table. The DDL commands are explained in the next section.
- Data manipulation language (DML): The purpose of data manipulation is twofold. First, it has commands to input, modify and delete the data in the SQL tables. Second, it has the SELECT command, which helps in the retrieval of data from one or multiple tables.
- Data control language (DCL): The purpose of these commands is to specify user's privileges to database users. These commands are very important in client/server databases with different types of users.

7.3 DATA DEFINITION LANGUAGE

The design of a database consists of the physical, conceptual, and external schema. To implement this design, you need to define these schemas using SQL. In this unit, we use commands to define the conceptual schema. The external schema-related commands are discussed in the next unit. As far as the physical schema is concerned, we will cover only a few commands. The conceptual schema, which

includes the relational schema and attributes, is implemented in RDBMS as a set of tables and columns, whereas the tuples of the relations are implemented with the help of rows or data records.

We will explain some of the basic DDL commands in this section with the help of an example. Consider two relations of a UNIVERSITY database system, namely STUDENT and PROGRAMME.

The student relation (STUDENT) has the following data: student ID (STID), which is also the primary key, the name of the student (STNAME), the programme code in which that student is registered (PROGCODE) and the mobile phone number of the student (STMOBILE). Please note that this schema assumes that a student is allowed to register in only a single programme for which s/he is allotted a unique student ID.

The second relation, PROGRAMME, has the following data: programme code (PROGCODE), which is also the primary key, the name of the programme (PROGNAME) and the fee for that programme.

STUDENT (STID, STNAME, PROGCODE, STMOBILE) and
PROGRAMME (PROGCODE, PROGNAME, FEE)

The first step would be to define the datatypes of various attributes. We propose to use the following data types for the attributes.

	Relation Name	STUDENT		
Attribute	<u>STID</u>	STNAME	PROGCODE	STMOBILE
Data type	Character	Character	Character	Number
Length	4	40	6	12 digits
Constraint	PRIMARY KEY	NOT NULL	FOREIGN KEY References PROGRAMME table	-

	Relation Name	PROGRAMME	
Attribute	<u>PROGCODE</u>	PROGNAME	FEE
Data type	Character	Character	DECIMAL
Length	6	30	5
Constraint	PRIMARY KEY	NOT NULL	>1000 and < 50000

Figure 1: Example Relations

Once you have completed the data design, the next step would be to use SQL to create the SQL tables. To do so, you should learn about different data types in SQL, commands for creating tables in SQL, commands for creating constraints, etc. Let us discuss each of these.

Data Types in SQL: SQL supports many data types. Figure 2 lists some of the commonly used data types of SQL. For more data types supported by a DBMS, you may refer to the information manual of that DBMS. You may select one of these data types for each column.

CHAR (<i>n</i>)	It accepts a character string of size <i>n</i> . The character string can include alphabets, numeric digits, and special characters. The size of each string is fixed, that is, <i>n</i> .
-------------------	--

VARCHAR (<i>n</i>)	It accepts a character string up to the size <i>n</i> . The character string can include alphabets, numeric digits, and special characters.
BOOLEAN	It accepts a value False (zero value) or True (non-zero value)
INTEGER (<i>n</i>)	It can store signed integers or unsigned integers of length 32 bits. <i>n</i> represents the display width of the integer.
DECIMAL (<i>n</i> , <i>d</i>)	It can store decimal numbers of size <i>n</i> having <i>d</i> digits after the decimal point. The default value of <i>d</i> is 0.
DATE	Represents a valid date. It uses 4 digits for years and 2 digits each for month and day.
TIMESTAMP	It assigns a timestamp, which can be used to determine the recency of data.

Figure 2: Some Basic Data Types

For the relations of Figure 1, you may select CHAR for STID, PROGCODE, STMOBILE (as it is not required for computation), and VARCHAR for STNAME and PROGNAME, as the names of students may be of different lengths; so are the names of the programme's. The data type of FEE is DECIMAL(5).

Creating the Database and the Tables: In order to create the tables as given in Figure 1, you need to first create a database using the following command:

```
CREATE DATABASE <name of the database>;
```

```
USE <name of the database>; //This command is needed when the DBMS has multiple databases.
```

The following commands will create the UNIVERSITY database:

```
CREATE DATABASE UNIVERSITY;
```

```
USE UNIVERSITY;
```

Now, you can create the tables using the create table command. The following is the syntax of this command:

```
CREATE TABLE <name of the table> (
    ColumnName1 <data type> [constraints],
    ColumnName2 <data type> [constraints],
    ...
);
```

The following are the descriptions of the create table command:

- The name of a table should begin with an alphabet. It should not contain blank space and special characters except the underscore character (_).
- You should not use reserved words as a name of a table.
- You should use a unique name for each column in a table.
- The constraints are optional and therefore, shown in [] brackets.
- The data type should include the size of the column.
- You may use any of the following constraints on the column:

NOT NULL	This column of a table cannot be left blank while data entry or modification.
UNIQUE	This value should be unique across all the values in this column.
PRIMARY KEY	The column is or is a part of the primary key.
CHECK	It is followed by certain conditions, which should be fulfilled by the column.
DEFAULT	When a default value is specified for a column.
REFERENCES	This is used for specifying referential constraints, while creating a table.

Figure 3: Some of the constraints in Create Table Command

Now, you are ready to create the tables. Since the table STUDENT contains a reference to PROGRAMME table, therefore, you may create the PROGRAMME table first using the following command:

```
CREATE TABLE PROGRAMME
(
    PROGCODE CHAR(6) PRIMARY KEY,
    PROGNAME VARCHAR(30) NOT NULL,
    FEE DECIMAL (5),
    CHECK (FEE>1000 AND FEE<50000)
);
```

Now, you are ready to create the STUDENT table. You will use the following command to create the table.

```
CREATE TABLE STUDENT
(
    STID CHAR(4) PRIMARY KEY,
    STNAME VARCHAR(40) NOT NULL,
    PROGCODE CHAR (6),
    STMOBILE CHAR (12),
    FOREIGN KEY PROGCODE REFERENCES PROGRAMME (PROGCODE)
    ON DELETE RESTRICT
);
```

The Referential Action: Please note that in the command of creating the STUDENT table, we have used a referential action RESTRICT, which will make sure that any deletion of a record in PROGRAMME table will be restricted in case even one record of STUDENT table contains that PROGCODE. Thus, this action will ensure that there is no violation of the referential integrity constraint during deletion of a record from the PROGRAMME table. The other possible referential action is CASCADE. In this case, the deletion of a record in PROGRAMME table will result in the deletion of all the records of all the students whose PROGCODE is the same as the record, which is getting deleted in the PROGRAMME table.

Creating an Index: You may notice that the primary key of the STUDENT table is STID, therefore, the STUDENT table would be organised in the order of STID. However, many database queries may require the student records in the order of PROGCODE. This would require you to create an index on PROGCODE in the STUDENT table to enhance the query performance. The following command can be used to create an index:

```
CREATE [UNIQUE] INDEX <name of the index>
ON <name of the table> (ColumnName1, ColumnName2, ...)
```

You use the keyword UNIQUE, only if a unique index is to be created. For the given example, you may create the following index for the STUDENT table.

```
CREATE INDEX PROGINDEX ON STUDENT (PROGCODE);
```

Other DDL commands: There are a large number of DDL commands. We will discuss only a few commands here. You may refer to the DBMS documentation for more commands.

Commands to alter a Table: For altering a table, you may use an ALTER TABLE command. This command can be used for performing the following functions:

- You can add a new column to an existing table, or you can modify a column of an existing table, by using the command:
ALTER TABLE <name of the table> ADD/MODIFY (ColumnName1, <datatype>, ...);
- You can add a new constraint in a table using the following command:
ALTER TABLE <name of the table> ADD CONSTRAINT
 <name of the constraint> <type of the constraint> (ColumnName);
- You can drop a constraint on a table or enable or disable it using the following command:
ALTER TABLE <name of the table> DROP/ENABLE/DISABLE <name of the constraint>;

Commands to delete a Table or an Index: You can remove a table or an index, which is not required any more. The following commands can be used for these purposes.

DROP TABLE<name of the table>;
DROP INDEX <name of the Index> ON <name of the table>;

Commands to create a Domain: As defined earlier, SQL has a large set of data types. However, in many database implementations, you need to create a more meaningful domain that can define the data types and constraints on the data of a specific attribute or column. The following command can be used to create domains. It may be noted that the following command may not be defined in many DBMSs.

CREATE DOMAIN <Name of the Domain> AS <Data type> CHECK (<Constraints>);

You may refer to the DBMS documentation for more details.

Check Your Progress 1

- 1) List the advantages and disadvantages of using SQL.

.....

.....

.....

.....

- 2) Consider a hotel that has three tables: ROOM (RNo, RType, RRent), BOOKING (CustID, RNo, BookedFrom, BookedTo) and CUSTOMER (CustID, CustName, CustAddress, CustPhone). Assume the structure of these tables and create the tables using SQL. You may assume the following constraints for the HOTEL database:

- The type of hotel rooms can be: Single Room and Double Room, default room type should be Single room.
- The room rent is between 5000 per day to 25000 per day.
- Rooms are on different floors and numbered from 101 to 150.
- While booking the room the BookedFrom and BookedTo should follow the following relationship: Today's Date <= BookedFrom <= BookedTo
- No room should be booked twice for a specific date.

7.4 DATA MANIPULATION LANGUAGE

Once you have created a database and database tables along with the necessary constraints, the next step is to insert data in the tables. While inserting the data in the tables, you may commit some mistakes or there may be the need of making certain changes in the data of the tables, therefore, you would be requiring SQL commands to INSERT, UPDATE and DELETE records in a database table. These are Data manipulation language (DML) commands. These DML commands allow you to input and edit the data in the tables. Further, you would like to retrieve selected information from the database. In this section, first, we discuss the command to insert, update or delete the data followed by commands to retrieve information from the database. You may please note that the changes made by the DML statements are made permanent only after these operations are COMMITTED. You will learn about COMMIT in Unit 9.

7.4.1 Data Insertion, Updating and Deletion

The DML commands are used for inputting and editing data in a database table. To insert data in a table, you may use the insert command, which is explained next.

Inserting Data: The following command is used to insert a record into a table. The following two formats of insert commands are used:

If you are inserting a record that had data for all the columns, then you can simply use the command format:

```
INSERT INTO <name of the table> VALUES (v1, v2, v3, ...);
```

Please note that v1, v2, etc. are the values that are to be inserted into the respective column of the database. For example, to insert data into the PROGRAMME table, you can use the following INSERT command:

```
INSERT INTO PROGRAMME  
VALUES ("PGDCA", "Postgraduate Diploma in Computer Applications", 22000);
```

Please note the following with respect to the insert command, given above:

- The values inserted into the table would be PROPCODE as *PGDCA*; PROGNAME as *Postgraduate Diploma in Computer Applications*; and FEE as *22000*.
- You can use parameters instead of actual values, for example, you can use INSERT INTO PROGRAMME VALUES (&1, &2, &3); These parameter values can be input at the time of execution of the query.
- You can use a sub-query (which will be explained in the next unit) instead of the values given in the command in the (...).

However, if you do not want to insert values in all the columns of the table, then you need to use the following format:

```
INSERT INTO <name of the table> (C1, C2, C3, ..., Cn) VALUES (v1, v2, v3, ..., vn);
```

Here, C1, C2, ... represents the column names and v1, v2, ... represents the corresponding value of a column. Please note that you need to specify n values for the n columns. For example, suppose you do not know the mobile number of a new student, still his/her information can be entered in the STUDENT table using the following SQL command:

```
INSERT INTO STUDENT (STID, STNAME, PROGCODE)
VALUES ("1001", "RAMESH SHARMA", "PGDCA");
```

Please note the following with respect to the insert command, given above:

- The insertion is possible as STMOBILE has no constraint. It can be left blank.
- The values inserted into the table would be STID as *1001*; STNAME as *RAMESH SHARMA*; PROGCODE as *PGDCA*; and STMOBILE as NULL.
- Please also note that the above insert statement will not cause a violation of foreign key constraint, as we have already inserted the PGDCA programme details in the PROGRAMME table.

Updating Data: For updating data, you may use the update command of SQL. The format of the command is given below:

```
UPDATE <name of the table>
SET <C1> = <v1>
WHERE <conditional statement>;
```

For example, to update the mobile number data of student 1001 to the number, say 8484848484, you can use the command:

```
UPDATE STUDENT
SET STMOBILE = "8484848484"
WHERE STID = "1001";
```

You can also use a subquery in an update statement (subqueries are covered in unit 8). For example, consider the fee of all the programmes, which has more than 100 students, is raised by 5%, then the following update command may be used to update the PROGRAMME table.

```
UPDATE PROGRAMME
SET FEE = FEE * 1.05
WHERE PROGRAMME.PROGCODE IN (
    SELECT STUDENT.PROGCODE
    FROM STUDENT
    GROUP BY STUDENT.PROGCODE
    HAVING COUNT(*) > 100);
```

The purpose of this subquery will be clear after you go through the next subsection. The subquery will find those programmes which have more than 100 students.

Deleting Data: You can use the following command to delete one or more records from a table:

```
DELETE FROM <name of the table>
WHERE <conditional statement>;
```

The delete command may delete one or more data records at a time. For example, you can try deleting the data of the PGDCA programme from your database, using the following command.

```
DELETE FROM PROGRAMME
WHERE PROGCODE= "PGDCA";
```

However, as there exists a foreign key constraint with referential action RESTRICT and you have already inserted a student who has PGDCA as his programme, therefore, DBMS will not allow you to delete the programme PGDCA from the PROGRAMME table. In case, you want to do so you need to first delete records of all the students of PGDCA from the STUDENT table and then you can delete the PGDCA programme from the PROGRAMME table. Please note that you can use subqueries instead of conditional statement in deletion also.

7.4.2 Data Retrieval

One of the most popular features of any DBMS is the ad-hoc query facility, which requires data retrieval as per the need and access rights of the user. SELECT statement is one of the most used statements of DML, as it helps in the retrieval of requisite data. In this section, we discuss various clauses of this statement.

SELECT Statement: The following is the basic format of the select statement.

```
SELECT      <List of Column names or expressions to be displayed>
FROM        <List of Tables that contain the data, whose columns are in select>
WHERE       <Conditions for selection of records for display>;
```

The following are examples of the use of this statement for the retrieval of data from the two relations given in Figure 1.

Example 1: List the details of all the programmes of the University.

For answering this query, are using one wildcard character (*), which represents all the columns of a table. Please note that in case, you have used * in an arithmetic expression in the SELECT clause, it will be treated as a multiplication sign.

```
SELECT      *
FROM        PROGRAMME;
```

This statement will display the PROGCODE, PROGNAME and FEE of all the records in the PROGRAMME table.

Example 2: List the programme code and programme names of all the programmes of the university, whose fee is less than or equal to 5000.

```
SELECT      PROGCODE, PROGNAME
FROM        PROGRAMME
WHERE       FEE <= 5000 ;
```

Please note the select clause now contains only those column names, which are to be displayed.

Example 3: List the id, name and mobile number of all the PGDCA students.

```
SELECT      STID, STNAME, STMOBILE
FROM        STUDENT
WHERE       PROGCODE = "PGDCA" ;
```

Example 4: This example shows the use of the **arithmetic operator** and **alias**. What would be the fee of the PGDCA programme, if students are given a discount of 15%?

```
SELECT      PROGCODE, PROGNAME, FEE*0.85 AS DISCOUNTEDFEE
FROM        PROGRAMME
WHERE       PROGCODE = "PGDCA" ;
```

Please note the computation of the fee and assigning the result of the computation a new name, that is an alias, DISCOUNTEDFEE. Please note that the syntax of assigning an alias may be different in different RDBMS.

Even in SQL expressions, the operator precedence is followed. Further, these expressions are executed from the left towards the right.

Example 5: This example shows the use of the **concatenation operator**. If you want to display the names of the programme as: “PGDCA: Postgraduate Diploma in Computer Applications” programme, you should use the following SQL command.

```
SELECT      PROGCODE + “:” + PROGNAME
FROM        PROGRAMME
WHERE       PROGCODE = “PGDCA” ;
```

Please note that in this statement the concatenate operator is ‘+’. However, in different DBMSs it may be different, for example, MySQL uses CONCAT() function, whereas Oracle uses || as a concatenation operator. In addition, please also note the use of the literal character string “:” in the command. A literal string may not be part of any column value but can be added to enhance information display.

Example 6: This example demonstrates how duplicate rows can be eliminated using the **DISTINCT** operator. Find the programme code of those programmes, which has at least one student.

To find the answer to this query, you may use the STUDENT table and Project it on programme code.

```
SELECT      DISTINCT PROGCODE
FROM        STUDENT
```

In case you do not use DISTINCT then you will get duplicate values of programme code.

Example 7: This example demonstrates the use of the range operator **BETWEEN ... AND**. To find the list of programmes whose fee is >=5000 but <= 15000, you may use the following command:

```
SELECT      *
FROM        PROGRAMME
WHERE       FEE BETWEEN 5000 AND 15000;
```

Please note that both the values, viz. 5000 and 15000 are included in the range.

Example 8: This example demonstrates the use of the set operator **IN**. Find the students of PGDCA or BCA programmes. One way of answering that query would be:

```
SELECT      *
FROM        STUDENT
WHERE       PROGCODE IN (“BCA”, “PGDCA”) ;
```

Example 9: This example demonstrates the use of **LIKE** operator for matching a pattern of characters in the columns which are of CHAR or VARCHAR type. It may be noted that LIKE operator is supported by several wildcard characters, which may be different in different DBMS. In this example, we demonstrate the use of % wildcard character that matches zero or many characters in a string. For example, a string like %COM% will match with strings: **COMPUTER**, **COMMERCE**, **INCOME**, **MCOM** etc.

To find the list of all the programmes, which have word “Computer”, you may give the command:

```
SELECT      *
FROM        PROGRAMME
WHERE       PROGNAME LIKE “%Computer%”;
```

Example 10: This example demonstrates the use of **IS NULL** operator. Find the list of students, whose mobile number is not with the University.

```
SELECT      *
FROM        STUDENT
WHERE       STMOBILE IS NULL ;
```

Example 11: SQL uses the logical operators, i.e., NOT, AND and OR. The precedence of these operators is shown in the following table:

Operators in increasing order of Precedence
Comparison operators (e.g. <, =, > etc.)
NOT
AND
OR

To print the name of all the students of PGDCA or BCA can also be written as:

```
SELECT      STID, STNAME
FROM        STUDENT
WHERE       PROGCODE = "BCA" OR PROGCODE = "PGDCA" ;
```

Ordering of Results using ORDER BY clause

The SELECT statement of SQL, in addition to SELECT, FROM and WHERE clauses, supports three more clauses, viz. ORDER BY clause, GROUP BY clause and HAVING clause. We will discuss the ORDER BY clause here and the remaining two clauses are discussed in the next section.

The ORDER BY clause is used, when you want to display the records in the sorted sequence of one or more columns. This sorted order can be increasing or decreasing. Also, you can use more than one column for sorting the data. Please note the following points about ORDER BY clause:

- ORDER BY is the last clause of a SELECT statement.
- You can mention ASC for ascending order or DESC for descending order. In case you do not mention any order, then ordering, by default, is in ascending order.
- You can sort the records on a column, which is part of the table given in FROM statement, even if you have not displayed that column.
- You can also sort on the alias that has been created by you in the select statement.

Example 12: List the name of the students in the order of programme and within a programme in alphabetical order.

```
SELECT      PROGCODE, STNAME
FROM        STUDENT
ORDER BY    PROGCODE, STNAME ;
```

Check Your Progress 2

- 1) Write the SQL commands to insert the following data in the STUDENT and PROGRAMME table. Highlight the errors, if any.

Relation Name	PROGRAMME	
PROGCODE	PROGNAME	FEE
BCA	Bachelor of Computer Applications	48000
MCA	Master of Computer Applications	60000

Relation Name	STUDENT		
STID	STNAME	PROGCODE	STMOBILE
1002	Abc	BCA	
1003	NULL	PGDCA	91999999999
1004	XYZ	MCA	

- 2) List the various clauses of the SELECT statement giving their purpose.

- 3) Consider the following two relations – S and SP.

S

SupplierNo	SupplierName	SupplierRanking	SupplierCity
S1	ABC	60	Delhi
S2	DEF	30	Mumbai
S3	EFG	50	Bangaluru
S4	FGH	40	Chennai
S5	GHI	NULL	Delhi

SP

SupplierNo	PartNo	QuantitySold
S1	PA	1000
S2	PA	500
S2	PB	1200
S3	PB	700
S5	PA	2100
S5	PB	1200

- Find the name of the suppliers, whose ranking is lower than 40 and the city of the supplier is Chennai.
- List the names of the suppliers of Delhi city in the decreasing order of the supplier ranking.
- List the names of the supplier who have supplied the part PB (use IN operator).
- List the codes of all the suppliers who have supplied at least one part.

- e) List all the suppliers, who have “EF” in their names.
 - f) Get part numbers for parts whose quantity sold in a supply is greater than 1000 or are supplied by S2. (Hint: It is a retrieval using union).
 - g) List the names of the suppliers, whose ranking is not given.
-
-

7.5 GROUP BY CLAUSE AND AGGREGATE FUNCTIONS

In the previous section, you have gone through the concept of data manipulation language (DML). We have discussed the SELECT statement and its various clauses. In a database system, several queries require DBMS to produce information about a group. For example, you may be interested in finding the average marks of the group of PGDCA students vis-à-vis BCA students. The SQL supports a GROUP BY clause for such cases. In addition, SQL also supports a number of functions that can find aggregate information for a group of records. These functions are required to find the sum, average, counting of records etc. These are called aggregate functions. The following table defines some of the important aggregate functions used in SQL.

count	Used to count the number of records
sum	Finds the sum of the data of a column
avg	Finds the average of the data of a column
max	Finds the maximum value from the data of a column
min	Find the minimum value from the data of a column

Figure 3: Some Aggregate functions in SQL

Let us explain the use of these functions with the help of a few examples.

Example 13: Find the number of students of PGDCA.

```
SELECT      COUNT(*)
FROM        STUDENT
WHERE       PROGCODE = “PGDCA”;
```

Please note that the wildcard *, in this case, indicates all the records to be counted, which fulfil the WHERE condition.

Example 14: Find the minimum, maximum and average fees of all the programmes.

```
SELECT      MIN(FEE), MAX(FEE), AVG(FEE)
FROM        PROGRAMME
```

GROUP BY clause

GROUP BY clause can be used to group records on certain criteria, e.g., you can group students on the basis of their Programmes. This clause is added after WHERE clause in the SELECT statement. In a SELECT statement in which you have used a GROUP BY clause, you can only use the aggregate functions or the column name on which you have grouped the data in the SELECT clause. The following example demonstrates this aspect:

Example 15: Find the number of students in every programme of the University.

You can answer this query by grouping the records on PROPCODE and finding the count of the student ids in each group.

```
SELECT      PROPCODE, COUNT(STID)
FROM        STUDENT
GROUP BY    PROPCODE;
```

Please note that in the SELECT clause of the SELECT statement above, we have used only the PROPCODE and the aggregate function COUNT.

HAVING clause

The HAVING clause can be used to specify a condition for a group. It is different from the WHERE clause, which is applicable for all the records, whereas the condition specified in the HAVING clause is to be fulfilled by a group of data. The following example explains the use of the HAVING clause.

Example 15: Count the number of students in each programme, where the mobile number of students is not given. List only those programmes which have more than 5 such Students.

```
SELECT      PROPCODE, COUNT(STID)
FROM        STUDENT
WHERE       STMOBILE IS NULL
GROUP BY    PROPCODE
HAVING      COUNT(STID) < 5;
```

Please note that in the SELECT clause of the SELECT statement above, we have used only the PROPCODE and the aggregate function COUNT.

7.6 DATA CONTROL LANGUAGE

The purpose of data control language (DCL) is to create users and assign access rights to them. In general, these commands are executed by a database administrator. The following are some of the most used DCL commands.

Creating a new user: You can create a new user using the following command:

```
CREATE USER <username for database user> IDENTIFIED BY <Password for the user>
```

For example, you can create a new user with the username “PGDCA_Student” with the password “PGDCA123”

```
CREATE USER PGDCA_Student IDENTIFIED BY PGDCA123
```

Use of GRANT Command: GRANT is used to give different kinds of accesses to a database user. Block3 covers the basic aspects of the GRANT option. In general, SQL supports two kinds of access permissions:

- The permissions that are at the system level.
- The permissions at the level of an object, such as a table, record, column etc.

The system-level permissions are, in general, specific to the DBMS environment, therefore, you may refer to the system documentation for details on such permissions. In this section, we provide a basic introduction to object-level permissions with the help of examples.

To give permission to get information from a table, the following SQL command may be used.

```
GRANT SELECT ON STUDENT, PROGRAMME TO PGDCA_Student;
```

To give permission for inserting and updating a record in the STUDENT table, you may use the following SQL command:

```
GRANT INSERT, UPDATE ON STUDENT TO PGDCA_Student;
```

In case you want to GRANT the SELECT access rights to more than one user on STUDENT table, then you may use the following command:

```
GRANT SELECT ON STUDENT TO PGDCA_Student1, PGDCA_Student2;
```

Use of REVOKE command: The REVOKE command is used to remove the access permissions that were granted to a user.

For example, you can also revoke SELECT permission using the following command:

```
REVOKE SELECT ON STUDENT FROM PGDCA_Student;
```

The following command will revoke all permissions of a user on STUDENT table.

```
REVOKE ALL ON STUDENT FROM PGDCA_Student;
```

Use of DROP command: You can remove a user using the DROP command.

For example, you can drop PGDCA_Student using the following SQL command:

```
DROP USER PGDCA_Student;
```

Check Your Progress 3

Consider the supplier relations given in Question 3 of Check Your Progress 2.

1) Write the SQL commands for the following aggregate operations.

- Find the total supply of part PA.
- Count the number of suppliers who have made a supply.
- List the part number and the total quantity supplied for that part.

.....
.....

2) Write the SQL commands for the following queries:

- List the suppliers who have made more than one supply.
- List the suppliers who have made more than one supply, with each supply being more than 1000.
- Find the part number and maximum quantity supplied, for the parts for which the total quantity supplied for that part is more than 1000. You may order the result in ascending order of part numbers.

.....
.....

3) Write the SQL commands for the following:

- Create a new user ABC having the password XYZ.
- Grant the user ABC to read table S only.
- Cancel all the access permissions of ABC on table S.

7.7 SUMMARY

SQL is one of the most important languages for any RDBMS. It allows a user to create tables, insert and update data in the tables and retrieve information from the tables. In addition, data of a table can be made available to only authorised users. This unit first introduces you to basic aspects of SQL and thereafter discusses the DDL commands. It is important that while creating tables you also include the constraints that are to be fulfilled by the tables. The constraints in SQL may be implemented using CHECK clause, PRIMARY KEY clause, FOREIGN KEY clause etc. You must also implement the referential action in case of referential integrity. You can also alter the tables if needed. This unit discusses the DDL commands for all the above operations. After creating the table using the DDL commands, next you use the DML commands to insert data into the tables. In case of any changes in the data values in the table, you may use the UPDATE command of DML. The commands for data insertion and updating have been discussed in the unit. One of the most important DML commands is SELECT which allows the retrieval of data from the table based on various criteria. This unit discusses various clauses of SELECT statement, viz., SELECT, FROM, WHERE, GROUP BY, HAVING and ORDER BY clauses. Finally, the unit discusses the CREATE USER, GRANT, REVOKE and DROP commands of DCL. You may please note that this unit does not cover all the SQL commands. You must practice these commands and learn more SQL commands from the user documentation of DBMS that you may use.

7.8 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) The following are the advantages of using SQL:
 - It is a standard query language across RDBMSs; therefore, you can port your data from one DBMS to another. Also, you just need to master SQL to use any RDBMS.
 - SQL is more English-like, so is easy to learn and use.
 - SQL exists for both interactive environments and as embedded SQL. Thus, you can use SQL for ad-hoc queries as well as in programs, like web-based programs.Some of the disadvantages of SQL are:
 - A query can be implemented in many ways, this may confuse a user.
 - The language, in its present form, is very large.
 - Some of the functions of the SQL are not portable across DBMS.
 - The result of an SQL command may allow duplicates, which are not relational in nature.
- 2) In this implementation, a lot of constraints have been defined, so instead of directly putting them in tables, first, we define the domains and use these domains in the CREATE table command. This will be a neater and more maintainable implementation.
`CREATE DOMAIN TYPEOFROOM AS CHAR (1) CHECK (VALUE IN (S, D));`

$$);$$

//Please note this constraint may not work on certain DBMS, where NOT EXISTS clause is not supported.

Check Your Progress 2

- 1) You may use the following sequence of instructions:

```
INSERT INTO PROGRAMME
```

```
VALUES ("BCA", "Bachelor of Computer Applications", 48000);
```

The above insertion will be successful.

```
INSERT INTO PROGRAMME
```

```
VALUES ("MCA", "Master of Computer Applications", 60000);
```

The above insertion will fail, as the FEE is more than the allowable fee value.

```
INSERT INTO STUDENT (STID, STNAME, PROGCODE)
```

```
VALUES ("1002", "Abc", "PGDCA");
```

The above insertion will be successful, as PGDCA was inserted in Section 7.4.1

```
INSERT INTO STUDENT (STID, PROGCODE, STMOBILE)
```

```
VALUES ("1003", "BCA", "9199999999");
```

The above insertion will fail, as the Student name column value must be given, as it has a constraint NOT NULL.

```
INSERT INTO STUDENT (STID, STNAME, PROGCODE)
```

```
VALUES ("1003", "XYZ", "MCA");
```

The above insertion will fail, as the MCA programme insertion had failed earlier, so it will result in a violation of the referential constraint.

- 2) The following are the basic clauses of SELECT statements.

SELECT is used to specify the columns or expressions that are to be displayed in the result.

FROM is used to list the tables that are to be used for data output.

WHERE is used to specify conditions for the selection of data for the display.

GROUP BY clause is used to specify the columns, whose values should be used to group the records of the table.

HAVING is used to specify conditions, which should be fulfilled by each group for selection for display.

ORDER BY is used to specify the ordering of the records for the output.

- 3) a)

```
SELECT SupplierName
FROM S
WHERE SupplierRanking < 40 AND SupplierCity = "Chennai";
```

The output of this will be:

SupplierName
FGH

- b)

```
SELECT SupplierName
FROM S
WHERE SupplierCity = "Delhi"
```

ORDER BY SupplierRanking DESC;
The output of this will be:

SupplierName
ABC
GHI

- c) SELECT SupplierName
FROM S
WHERE SupplierNo IN (SELECT DISTINCT SupplierNo
FROM SP
WHERE PartNo= "PB") ;

The output of this will be:

SupplierName
DEF
EFG
GHI

- d) SELECT DISTINCT SupplierID
FROM SP;
The output of this will be:

SupplierNo
S1
S2
S3
S5

- e) SELECT *
FROM S
WHERE SupplierName LIKE %EF% ;
The output of this will be:

SupplierNo	SupplierName	SupplierRanking	SupplierCity
S2	DEF	30	Mumbai
S3	EFG	50	Bangaluru

- f) SELECT DISTINCT PartNo
FROM SP X
WHERE X.QuantitySold > 1000
UNION
(SELECT DISTINCT PartNo
FROM SP Y
WHERE Y.SupplierNO = "S2") ;
The output of this will be:

PartNo

PA
PB

- g) `SELECT SupplierName`
`FROM S`
`WHERE SupplierRanking IS NULL ;`
The output of this will be:

SupplierName
GHI

Check Your Progress 3

1)

- a) `SELECT sum(QuantitySold)`
`FROM SP`
`WHERE PartNo = "PA" ;`
b) `SELECT count(DISTINCT SupplierNo)`
`FROM SP;`
c) `SELECT PartNo, sum(QuantitySold)`
`FROM SP`
`GROUP BY PartNo`

2)

- a) `SELECT SupplierNo`
`FROM SP`
`GROUP BY SupplierNo`
`HAVING Count(PartNo) > 1 ; // As each supply is of one part only.`
b) `SELECT SupplierNo`
`FROM SP`
`WHERE QuantitySold > 1000`
`GROUP BY SupplierNo`
`HAVING Count(PartNo) > 1 ; // As each supply is of one part only.`
c) `SELECT PartNo, Max(QuantitySold)`
`FROM SP`
`GROUP BY PartNo`
`HAVING Sum(QuantitySold) > 1000`
`ORDER BY PartNo;`

3)

- a) `CREATE USER ABC IDENTIFIED BY XYZ;`
b) `GRANT SELECT ON S TO ABC;`
c) `REVOKE ALL ON S FROM ABC;`